Modules 7–10: NumPy › Pandas › Matplotlib › Seaborn

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Analytics	M10-A1	4 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write your code in Python 3. For all coding tasks, submit your .py source file along with a screenshot of your program output. For chart-based tasks, save all charts as PNG files and include them in your submission folder. Label every task clearly with its section and task number. All work must be your own; AI-generated code is permitted only in Section D.

A Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question. Answers must demonstrate reasoning, comparison, or justification — not definition recall.

S 1

SCENARIO

You are building a delivery fee calculator for a food delivery platform. You have a NumPy array of 50,000 order distances (in km) and need to compute a delivery fee for every order simultaneously. A teammate suggests iterating over the array using a Python for-loop and appending results to a list.

Question: Explain why NumPy vectorisation is a technically superior approach to the for-loop suggestion. In your answer, describe what broadcasting allows when a single scalar fee rate (e.g., Rs 5 per km) is applied to the entire array, and identify one scenario where a Python loop would still be necessary despite NumPy being available.

S 2

SCENARIO

You are working on a reporting pipeline for a food delivery platform. You have a Pandas DataFrame with columns: order_id, restaurant_name, city, order_value, delivery_time_mins, and rating. A colleague proposes filtering the DataFrame in a for-loop once per restaurant and computing averages manually inside the loop.

Question: Explain why groupby() combined with agg() is the correct approach instead of the for-loop strategy. Describe the specific method chain you would write to compute average order value, average delivery time, and average rating per restaurant in a single operation, and explain what the resulting DataFrame's index represents after the operation.

S 3

SCENARIO

You are developing a monthly performance report for a food delivery startup. The report must present three pieces of information side by side: (a) total orders placed per month over a year, (b) the proportion of orders by cuisine type (Fast Food, Indian, Chinese, Desserts), and (c) the distribution of delivery times across all orders.

Question: Identify the most appropriate Matplotlib chart type for each of the three metrics and justify each choice based on the nature of the data. Then explain how plt.subplots() would be configured to arrange all three charts in a single figure, and state whether a shared x-axis would or would not be appropriate for this layout — and why.

S 4

SCENARIO

You are optimising a Matplotlib line chart that shows daily order volume for a food delivery app over a 90-day period. The chart renders correctly but stakeholders keep asking 'which day had the spike?' during every review meeting. The spike is the single highest-value day in the array.

Question: Describe how you would use plt.annotate() to programmatically identify and mark the maximum order day with an arrow pointing to the peak and a text label showing the day number and volume. Name at least two specific parameters you would pass to plt.annotate() and explain what each one controls in the rendered chart.

S 5

SCENARIO

You are designing a visual analysis of customer ratings for a food delivery app. Your dataset has 10,000 rows. Each row belongs to one of three restaurant categories (Fast Food, Fine Dining, Casual) and has a customer rating on a discrete 1–5 scale. Your manager wants to understand how ratings are distributed within each category and whether any category produces more extreme outliers.

Question: Compare sns.boxplot() and sns.violinplot() for this specific task. Which would you recommend and why? Provide at least two data-specific reasons that reference properties of this dataset (10,000 rows, 3 categories, discrete 1–5 scale), and explain one situation where the other chart type would be preferable.

S 6

SCENARIO

You are building a feature analysis report for a food delivery platform's data science team before a regression model is trained. Your DataFrame contains eight numeric columns: order_value, distance_km, delivery_time_mins, rating, discount_applied, peak_hour_flag, restaurant_age_years, and driver_experience_years.

Question: Explain how you would generate and interpret a Seaborn correlation heatmap to identify multicollinearity risks among these features. In your answer, describe what the colour scale in the heatmap represents, how to read a specific cell value, and what correlation magnitude threshold you would use to flag a potential multicollinearity problem — and why that threshold is conventionally chosen.

B Section B — Practical Coding Tasks

Complete all tasks in Python 3. Each task targets a specific module's concepts. Submit one .py file per task along with a screenshot of the console output or saved chart. Requirements must be met exactly — do not simplify or substitute.

Task 1: Delivery Fee Array Calculator (NumPy)

Build a NumPy-based program that computes and analyses delivery fees for a batch of food orders using vectorised operations — no Python loops.

- Generate a 1D NumPy array of 25 random delivery distances between 1.0 km and 15.0 km using np.random.uniform() with a fixed seed of 42.
- Compute the delivery fee for every order using the formula: fee = Rs 20 + Rs 5 × distance — applied as a single vectorised expression, not a loop.
- Use boolean indexing to extract and print all orders where the delivery fee exceeds Rs 60, showing both the distance and the fee for each qualifying order.
- Print the minimum, maximum, mean, and standard deviation of the complete fee array using NumPy statistical functions.

Task 2: Restaurant Performance Analyser (Pandas)

Build a Pandas program that creates a food delivery dataset and produces a filtered, ranked summary of restaurant performance using groupby aggregation.

- Construct a DataFrame with at least 40 rows containing columns: restaurant_name (5 unique names), city (3 unique cities), order_value, delivery_time_mins, rating (1.0–5.0), and cuisine_type (3 types).
- Use groupby() and agg() in a single method chain to compute: mean order value, mean delivery time, and mean rating — grouped by restaurant_name.

- Filter the grouped result to retain only restaurants with a mean rating above 4.0 and a mean delivery time below 35 minutes.
- Sort the final output by mean order value in descending order, reset the index, and print the complete result with column headers clearly visible.

Task 3: Monthly Order Trend Dashboard (Matplotlib)

Build a Matplotlib figure with three subplots that together form a visual performance report for a food delivery platform across a 12-month period.

- Generate synthetic monthly data using NumPy: 12 months of total orders (random integers 1,000–5,000), monthly revenue (orders × random average order value between Rs 200–Rs 400), and average delivery time (normally distributed, mean=28, std=4).
- Subplot 1 — Line chart: Plot total orders per month with circular markers; annotate each data point with its value displayed just above the marker.
- Subplot 2 — Bar chart: Plot monthly revenue; colour each bar green if revenue exceeds Rs 8,00,000 and red otherwise, and draw a horizontal dashed line at the Rs 8,00,000 threshold.
- Subplot 3 — Histogram: Plot 500 simulated delivery times (use the generated mean and std) with 15 bins; add a vertical dashed line at the sample mean labelled 'Mean'.
- Apply a figure title, individual subplot titles, axis labels on all subplots, and plt.tight_layout(); save the figure as food_delivery_dashboard.png at DPI 150.

Task 4: Full EDA Pipeline — Food Delivery Insights (NumPy

Build an end-to-end exploratory analysis pipeline that generates, cleans, enriches, and visualises a synthetic food delivery dataset using all four modules in sequence.

- Use NumPy (seed=7) to generate a 200-row dataset with columns: order_value (uniform Rs 100–Rs 800), distance_km (uniform 1–20), delivery_time_mins (normal ?=30, ?=8), rating (uniform 1.0–5.0), and discount_pct (uniform 0–30); load it into a Pandas DataFrame.
- Artificially introduce 5% null values into the delivery_time_mins and rating columns using np.random.choice(); then fill all nulls with the respective column medians.
- Add a derived column: delivery_speed_kmph = distance_km / (delivery_time_mins / 60); classify each row into a speed_band category ('Fast', 'Normal', 'Slow') using pd.qcut() on delivery_speed_kmph with 3 equal-frequency bins.
- Produce a Seaborn heatmap of the Pearson correlation matrix for all numeric columns with annotated values (annot=True) and a diverging colour palette; save as correlation_heatmap.png.
- Produce a Seaborn pairplot of order_value, distance_km, delivery_time_mins, and rating, coloured by speed_band; save as pairplot.png. Both charts must be saved at DPI >= 150 using plt.savefig().

C Section C — Mini Capstone Project

Mini Project: Food Delivery Analytics Console — Full EDA Report

Objective:

Build a menu-driven Python console program that performs a complete exploratory data analysis on a synthetic food delivery dataset, integrating NumPy array operations, Pandas data wrangling, Matplotlib charting, and Seaborn statistical visualisation into a single cohesive script.

Your project must:

- The program must present a numbered menu with at least 4 analysis options: (1) Summary Statistics, (2) Distribution Analysis, (3) Correlation Heatmap, (4) Restaurant Performance Report — and loop until the user chooses to exit.
- Each menu option must be implemented as a separate function; each function must use at least one technique from a different module (NumPy statistical functions, Pandas groupby, Matplotlib subplot, or Seaborn plot) — no function may duplicate another's primary technique.
- The underlying dataset must have at least 200 rows, generated with NumPy at program startup; the program must check for and fill any null values in numeric columns before any analysis option is invoked.
- All charts produced by the menu options must be saved as PNG files with descriptive filenames and DPI >= 150 — charts must not be displayed interactively during menu operation.
- After the user exits the menu, the program must print a plain-text summary report to the console listing: the top 3 restaurants by mean rating, the numeric column pair with the highest absolute Pearson correlation, and the overall mean and standard deviation of delivery time across the full dataset.

D Section D — AI-Augmented Learning

STEP 1 · BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a Python program that:

- Generates a synthetic food delivery dataset with at least 5 numeric columns and one categorical column (e.g., cuisine_type) using NumPy, loaded into a Pandas DataFrame.
- Produces a Seaborn pairplot of all numeric columns, coloured by the categorical column, and saves it as a PNG file.
- Computes the Pearson correlation matrix and displays it as a Seaborn heatmap with annotated cell values and a diverging colour palette.
- Exports the heatmap as a separate PNG file at DPI 200 with a descriptive filename.

STEP 2 · TEST & DEBUG (WITHOUT AI)

Then, working without AI, run the code on your machine and identify at least one bug, limitation, or improvement in the AI's solution — for example: missing DPI on savefig(), incorrect DataFrame dtypes, or a pairplot that breaks when the categorical column has null values. Fix the issue yourself.

SUBMIT All 3 items required

1. The exact prompt(s) you gave the AI tool.
2. The AI's original code and your corrected version (clearly marked).
3. A 3–4 line note explaining what you changed and why the AI's version needed the fix.